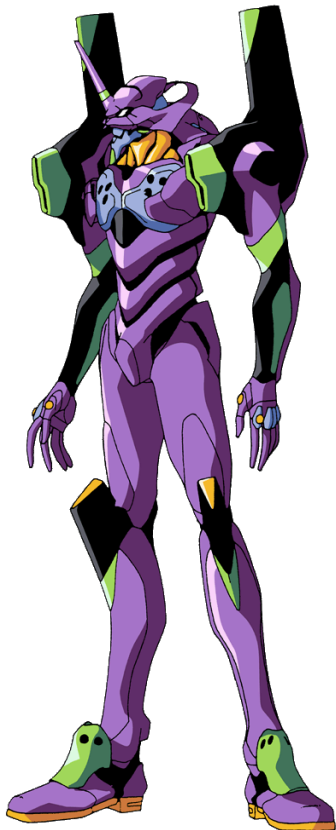


# Four Natural-Language Reasoning Systems

IMO25 · Rethlas · proofQED · Moonshine

# Evangelion Unit-01: Core, Armor, and Berserk Mode



|                              |                                                    |
|------------------------------|----------------------------------------------------|
| core / S <sup>2</sup> engine | the LLM – raw power, no direction of its own       |
| armor + restraints           | harness as RESTRAINT – keep it on task, no berserk |
| umbilical + weapons          | harness as POWER-UP – tools, retrieval, context    |
| pilot in entry plug          | the user's prompt / intent                         |
| Berserk (暴走)                 | the LLM off-leash – powerful, unsteered            |

Cut the harness and the core goes **Berserk**: huge output, no one steering – an LLM that walks straight off the task. The rig is what keeps it working *and* on course.

This session reads four systems – IMO25 · Rethlas · proofQED · Moonshine – as four rigs around the **same core**. The core never changes; all the engineering goes into the suit.

# The IMO25 Solver

[github.com/lyang36/IMO25](https://github.com/lyang36/IMO25)

MIT license – Lin Yang, Yichen Huang, 2025

Paper: **Gemini 2.5 Pro Capable of Winning Gold at IMO 2025**

arXiv:2507.15855

The run logs show end-to-end solutions to IMO 2025 problems 1–5, which the **authors describe** as gold-medal level. Problem 6 is not among them.

Core file: `code/agent.py`. A live demo can use the repo's own `run_logs/` – no API key needed.

One Python file calling one chat model:

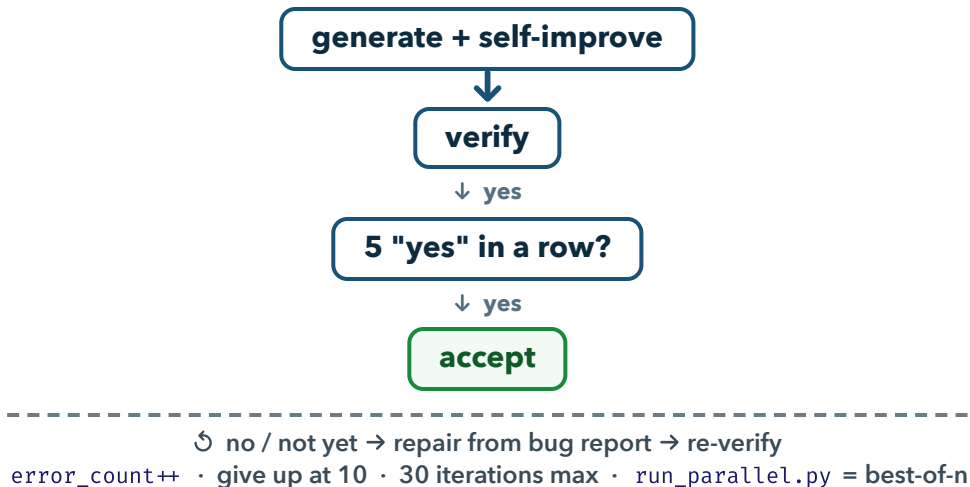
- no external tools, no retrieval;
- no database, no memory service;
- everything is prompts plus a small control loop.

Default model Gemini 2.5 Pro; siblings  
`agent_oai.py` (GPT-5), `agent_xai.py` (Grok-4).

# IMO25's Five Prompts and Control Loop

|                            |                        |
|----------------------------|------------------------|
| step1_prompt               | generation contract    |
| self_improvement_prompt    | one self-revision pass |
| verification_system_prompt | verifier role (grader) |
| verification_reminder      | verifier reminder      |
| correction_prompt          | feed bug report back   |

Generator and verifier are the **same model** under different system prompts (`verification_reminder` is the source's spelling). A single "yes" is not trusted – the verifier is stochastic – so accept only after **5 consecutive passes**; any failure resets the streak.



# IMO25: The Generation Contract (step1\_prompt)

- \* **Rigor is Paramount:** produce a complete and rigorously justified solution. A correct final answer derived from flawed or incomplete reasoning is considered a **failure**.
- \* **Honesty About Completeness:** if you cannot find a complete solution, **do not guess**; present only significant partial results you can rigorously prove.

Output Format:

1. Summary (Verdict + Method Sketch with precise lemma statements)
2. **Detailed Solution** (the full step-by-step proof only)

Then one self-improvement pass: *"review your solution carefully, **correct errors and fill justification gaps**, and follow the system prompt."*

The fixed output format forces a **separable artifact** – the Detailed Solution can be pulled out and graded on its own. "Honesty about completeness" is a prompt-level guardrail against confident fakes.

Verbatim from `step1_prompt` and `self_improvement_prompt` in `code/agent.py`.

# IMO25's Verification Prompt

You are an expert mathematician and a **meticulous grader** for an IMO-level exam. ... A solution is judged correct only if **every step is rigorously justified**.

Your sole task is to find and report all issues in the provided solution. You must act as a **verifier, NOT a solver**. **Do NOT attempt to correct** the errors or fill the gaps you find.

a. **Critical Error:**

any error that **breaks the logical chain** of the proof.

Fallacies (e.g. " $A > B$ ,  $C > D$ " implies " $A - C > B - D$ ") and factual errors (e.g. " $2 + 3 = 6$ ").

Procedure: explain it; do NOT check steps that rely on it; still scan independent parts (other cases).

b. **Justification Gap:**

the conclusion may be right, but the argument is **incomplete, hand-wavy**, or lacks rigor.

Procedure: explain the gap; ASSUME the step's conclusion; then verify all subsequent steps.

The verifier grades only the extracted **Detailed Solution**. Every issue is one of these two kinds – the taxonomy makes the verdict actionable, not a vague "looks wrong."

# The yes/no Verdict Gate

**long verification log**

->

**cheap yes/no call**

->

**good\_verify** ∈ {yes, no}

Response in "**yes**" or "**no**". Is the following statement saying the solution is correct, or does not contain critical error or a major justification gap?

<the verifier's Detailed Verification Log is appended here>

If the answer is not "yes", the verifier's Detailed Verification Log is kept as the **bug report** that drives repair.

Collapsing a long critique into one yes/no signal gives the control loop a clear branch condition – model judgment turned into a usable contract.

# The Repair Loop

```
solution = generate_then_self_improve(problem)
verify, good = verify_solution(problem, solution)
correct_count, error_count = 1, 0

for i in range(30):
    if good != "yes":
        solution = regenerate_with_bug_report( ... )
        correct_count = 0; error_count += 1
    verify, good = verify_solution(problem, solution)
    if good == "yes":
        correct_count += 1; error_count = 0
    if correct_count ≥ 5: return solution    # ACCEPT
    if error_count ≥ 10: return None        # GIVE UP
```

Below is the **bug report**. **If you agree** with an item, improve your solution so it is complete and rigorous. Note that the evaluator **can misunderstand** your solution and make mistakes. **If you do not agree, add detailed explanations** to avoid such misunderstanding.

The repair prompt allows for a **wrong verifier**: agree and fix, or disagree and clarify.



# Accept After 5 Consecutive Passes

**5x**

A single "yes" is not trusted: the verifier is itself a stochastic model.

```
if good == "yes": correct_count += 1
else:             correct_count = 0; error_count += 1

accept when correct_count ≥ 5
```

- Five consecutive passes → low false-accept risk.
- Any failure resets the streak.

# Bounded Retries and Best-of-n

## Bounded

`error_count`  $\geq 10 \rightarrow$   
abandon. 30 iterations max  
per attempt.

## Retried

Outer loop reruns the agent  
up to `max_runs` (default 10).

## Parallel

`run_parallel.py`: N agents  
(default 10), best-of-n.

State is saved every iteration ( `save_memory` ), so a long run is **resumable**.

# IMO25 Capabilities and Gaps

## Already has

- generator and verifier: the same model under two prompts;
- a separable artifact (fixed output format);
- an actionable verdict, then a yes/no gate;
- repair from feedback;
- accept after 5 consecutive passes;
- bounded, parallel, resumable.

## Still lacks

- machine-readable contracts;
- code-level guardrails;
- memory of failed attempts (keeps only the latest solution + verdict);
- named, inspectable skills;
- retrieval with applicability checks;
- verification as a reusable service;
- difficulty triage, an explicit plan, failure attribution.

In practice: reliable only on **short proofs ( $\approx 10$  pages)**; **40 min - 1 h** per proof with Gemini Deep Think; no state beyond the latest solution + verdict, so it cannot learn from failed attempts.

# Rethlas

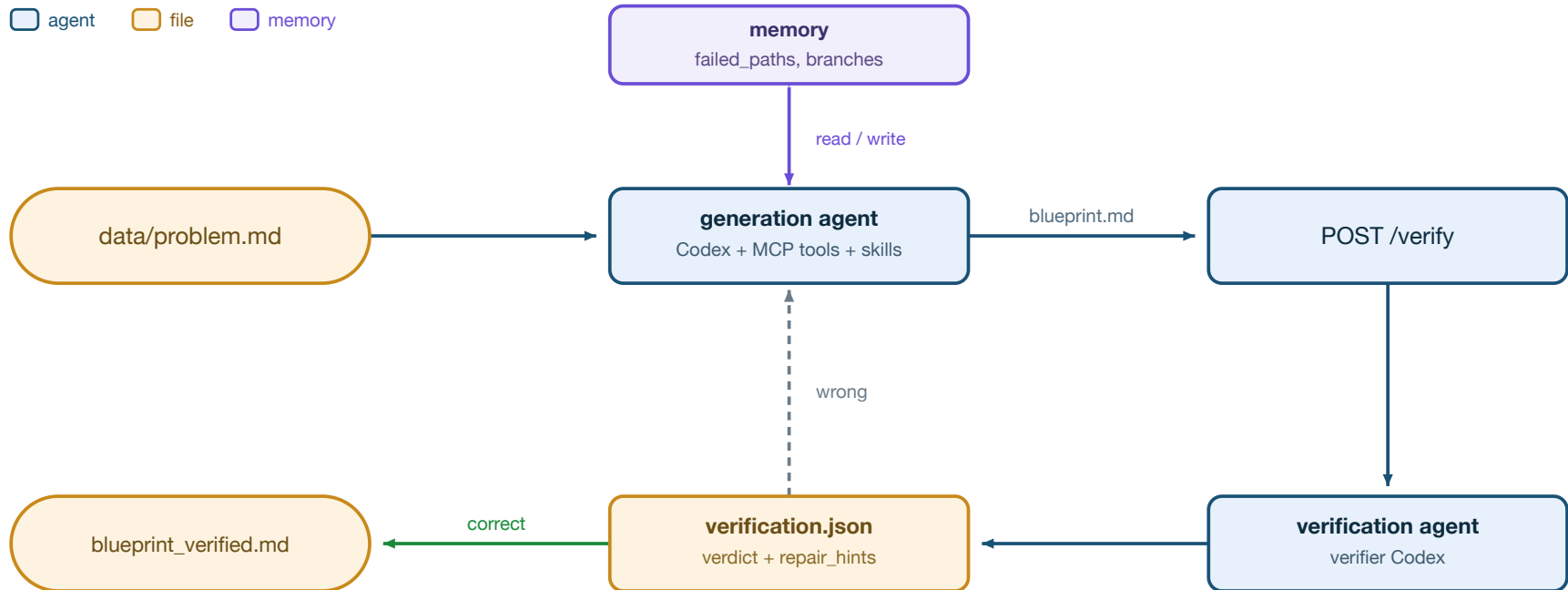
[github.com/frenzymath/Rethlas](https://github.com/frenzymath/Rethlas)

# The Loop Elements, IMO25 vs Rethlas

| loop element    | IMO25 (minimal)         | Rethlas (engineered)                |
|-----------------|-------------------------|-------------------------------------|
| role separation | two system prompts      | two workspaces + own files          |
| generator       | one chat thread         | Codex run with MCP tools + skills   |
| verifier        | a grading prompt        | verifier Codex behind /verify       |
| repair signal   | bug report text         | verification.json (verdict + hints) |
| memory          | flat file: latest only  | append-only channels + search       |
| acceptance      | 5 consecutive "yes"     | strict JSON verdict + blueprint     |
| audit           | log file + saved memory | results/ logs/ memory/ verify/      |

Each loop element becomes a concrete file, tool, or service in the Rethlas source.

# The Rethlas Run



# Two Agents: generator and verifier

An **agent** is an autonomous actor – its own instructions (`AGENTS.md`), tools, and memory – that decides its own next action toward a goal, not a function the orchestrator drives step by step.

## generator – explore

- chooses skills adaptively (no fixed chain)
- tools: `search_arxiv_theorems`, `memory_*`, `branch_update`
- writes the blueprint; repairs from the verifier's report, avoiding `failed_paths`
- never grades its own proof

## verifier – criticize

- reads the proof in textual order
- classifies each issue: **critical error** vs **gap**
- emits one strict, structured verdict
- never proves, repairs, or searches

The split lives in two workspaces – `agents/generation/`, `agents/verification/`, each with its own `AGENTS.md`, tools, memory, files, stopping rules. The directory is the mechanism, not the point.

# Contracts in AGENTS.md: files and formats

## generator → blueprint

```
in: data/<problem>.md  self-contained
   problem_id          namespaces outputs
out: results/{problem_id}/blueprint.md
     ... /blueprint_verified.md
```

blueprint.md – one block per result:

```
# lemma <label>
## statement
## proof
```

## verifier → JSON verdict

```
verification.json
verdict  "accepted" iff gaps=[]
        and critical_errors=[]
verification_report:
  summary
  checked_items[] {location,status}
  gaps[]          {location,issue}
  critical_errors[] {location,issue}
  external_reference_checks[]
  repair_hint     "" when accepted
```

The format *is* the API: a strict, machine-readable verdict gives the repair loop a branch condition. In IMO25 the same data was chat text parsed by string search.



# Launching a Run: run\_example.sh

```
1 config (env): PROBLEM_FILE, MODEL=gpt-5.4, REASONING_EFFORT=xhigh
2 guard: problem file must exist          → else exit 1
3 derive problem_id; mkdir logs/<rel>/
4 prompt = "Use AGENTS.md exactly to solve <PROBLEM_FILE>."
5 curl /health → if down: WARN "agent skips verification", continue
6 codex exec -C generation/ -m MODEL
    --dangerously-bypass-approvals-and-sandbox "<prompt>" (output → log)
7 report exit code + total time
```

The launcher just runs `codex exec`; the one-line prompt delegates **all** behaviour to `AGENTS.md`. The verifier is a separate service it only health-checks.

Only hard guard: **the file exists**. Path safety / "stay in the workspace" are instruction-level, not enforced here.

The verifier health check is **fail-open** (unreachable → warn + continue); the launcher runs with approvals/sandbox bypassed – fine for a trusted local checkout.

# Memory Channels and failed\_paths

```
memory/{problem_id}/<channel>.jsonl    append-only
```

```
immediate_conclusions  toy_examples  
counterexamples        big_decisions  (pivots)  
subgoals               proof_steps  
failed_paths           verification_reports  
branch_states          events
```

MCP: `memory_init` / `append` / `search`,  
`branch_update`, scoped by `problem_id`.  
`memory_search` = BM25 over the agent's **own  
appended records** (its scratchpad) → `{score,  
channel, item}` – not arXiv, not the truth store.

Key channel **failed\_paths**: an agent that forgets  
failures loops. `propose-...-plans` must read it first.

A long proof search exceeds one context window, so the state is externalized into **semantic channels**, not one flat transcript. Cost: append-only memory accumulates noise (wants status labels + compaction).

# When the Agent Searches Memory

Writing memory is only half of it – the channels are dead weight unless the agent **reads** them.  
`memory_search` is the read side; the rule is recall before generating.

## read-before-act

Most skills may search memory before new work – examples, proving, verify, ...

## explicit recall

`$query-memory`: do earlier conclusions / counterexamples / `failed_paths` inform this claim, subgoal, or branch? Test a claim against saved counterexamples.

## required first

`propose-...-plans` must read examples, counterexamples, `failed_paths`, branches **before** proposing – so it can't re-propose an abandoned route.

Disciplined: a query over the **smallest relevant channels** (not all), then summarize how the hits change the state; nothing useful → log `query_memory_stalled` and switch. Every query is an `events` record – recall is auditable.

# Skills as Policy Modules

Every skill is a `SKILL.md` with the **same contract**:

|                 |                           |
|-----------------|---------------------------|
| Input Contract  | what to read              |
| Procedure       | the steps to follow       |
| Output Contract | the artifact to write     |
| MCP Tools       | which tools it may call   |
| Failure Logging | what to record on failure |

The agent **chooses** skills from its state – named, inspectable policies replace one giant prompt.

**generation skills form a workflow**

|        |                                                                                                     |
|--------|-----------------------------------------------------------------------------------------------------|
| gather | immediate-conclusions, search-math-results, query-memory, toy/counterex.                            |
| plan   | propose-decomposition-plans<br>(needs the gathered info)                                            |
| prove  | direct-proving (screen, find stuck pts)<br>recursive-proving (1 sub-agent/plan,<br>after screening) |
| loop   | identify-key-failures → new plans                                                                   |
| verify | verify-proof (only when proof assembled)                                                            |

Verifier's own three: check-referenced-statements, verify-sequential-statements, synthesize-verification-report. Each skill's **Failure Logging** writes the memory channels (B6) – skills write memory, `memory_search` recalls.

# How the Agent Chooses Skills

No fixed chain. Each step: **assess state** → **choose** a skill by explicit conditions → **persist** its output to a channel → stop only when verification passes.

| stage  | skill                              | → writes                                                |
|--------|------------------------------------|---------------------------------------------------------|
| gather | conclusions /<br>examples / search | immediate_conclusions,<br>toy_examples, counterexamples |
| plan   | propose-plans                      | subgoals                                                |
| prove  | direct / recursive                 | proof_steps, branch_states                              |
| loop   | identify-failures                  | failed_paths                                            |
| verify | verify-proof                       | verification_reports                                    |

The discipline is in the **mandatory output fields**:

|                       |                                |
|-----------------------|--------------------------------|
| is_fragile            | conclusions self-flag risk     |
| uses_information_from | plans must cite prior evidence |
| migration_failures    | concrete failure attribution   |
| applicability_check   | no black-box citations         |

Closed loop: **identify-key-failures** → **failed\_paths** → **propose-plans** – the agent can't re-propose an abandoned route.

# The search-math-results Applicability Protocol

Tool: `search_arxiv_theorems`

The protocol **forbids** pasting a search hit straight into a proof – it must be checked first.

search-math-results requires:

- state the theorem completely
- read the proof when useful
- expand definitions from the paper
- check terminology matches THIS problem
- record source ids + why it applies

The same word means different things across papers; a theorem can fail by a hidden convention. A math agent needs more than RAG – an applicability protocol, enforced on **both** sides: the verifier re-checks every citation in `external_reference_checks[]`.

# The /verify Service

```
agents/verification/api/server.py
GET /health
POST /verify { statement, proof }
  → allocate run_id
  → build verifier prompt
  → codex exec inside verification/
  → write results/{run_id}/verification.json
  → return the JSON
```

The verifier is a **separate process**. The generator calls it over HTTP (`verify_proof_service`); `run_example.sh` only health-checks it (B5). The verdict it returns is the `verification.json` of B4 – not repeated here.

Why a service: verification becomes **callable** (re-check after every repair) and **replaceable** – a stricter, adversarial, or formal checker can be swapped in without touching the generator. If a capability matters, expose it as a narrow tool, not an informal "check your work."

# Sub-Agents After Decomposition

`recursive-proving` spawns sub-agents only after it has:

- decomposition plans;
- direct-proving reports;
- stuck points per plan;
- relevant failed paths and branches.

**Decompose** – a deliberate skill step  
`propose-...-plans`, run once  $\geq 2$  plans exist  
writes the plans  $\rightarrow$  `subgoals` channel

IMO25: best-of-n on **whole attempts** – "one try will work."

Rethlas: sub-agents on **decomposed subgoals** –  
"this splits into parts."

**Branch** – bookkeeping only  
`branch_update(problem_id, branch_id, state)`  
on a choice / backtrack  $\rightarrow$  `branch_states` channel

Decompose **creates** the alternatives; a branch is the **handle for pursuing one** (or abandoning it) – usually one branch per pursued plan. Decomposition first, so parallel branches don't duplicate confusion.



# Who Makes the Goal – Who Calls recursive-proving

`recursive-proving` does **not** create a goal. `propose-...-plans` already wrote each plan to the `subgoals` channel – a `goal` plus ordered subgoals. It only **distributes**: full target + one existing plan + the stuck points + "follow `AGENTS.md`".

**Who decides:** the main agent's adaptive loop in `AGENTS.md` – not a fixed order. The gate fires only when all three hold:

- every plan screened by `direct-proving`;
- none of them fully solved it;
- per-plan stuck points known → one sub-agent per plan.



---

Each sub-agent is a `subgoal-prover` running the same `AGENTS.md` – so it can recurse, bounded by `max_depth=3`, `max_threads=10`.

# branch vs subgoal: Two Scope Levels

Every memory record is tagged with three fields (schema from `obtain-immediate-conclusions`):

```
"scope": "global | branch | subgoal" // which level this record is about
"branch_id": "optional"              // which line of attack
"subgoal_id": "optional"             // which sub-statement
```

**subgoal** = a piece of **math content** – a sub-statement. An entry of a plan's `subgoals` array; referred to by `subgoal_id`.

**branch** = a **line of attack** (process). `branch_update` writes `{branch_id, state}` to `branch_states`, updated on a choice / backtrack.

Both ride on **every** record at once: `branch_id` × `subgoal_id` are **orthogonal tags** ("which line × which sub-statement") – not a parent/child tree.

---

The MCP server enforces none of this – `branch_update` stores arbitrary state; the structure is whatever the agent writes. "One branch per plan" is convention, not an invariant.

# results / logs / memory / verification Artifacts

blueprint

What proof did it produce?

verify JSON

What was accepted / rejected?

logs

What happened during the run?

memory

What can later steps reuse?

If the only artifact is the final answer, the system is hard to debug and trust – these surfaces give **observability**. Weakness: no manifest links blueprint → verifier report → logs → memory.

# Rethlas Limitations

- **Some guardrails stay instruction-level.** Path safety and "stay in the workspace" are prompt contracts, not code-enforced.
- **Verification health check fails open.** If a verified proof is required, the orchestration should fail closed.
- **Append-only memory accumulates noise.**  
Weak ideas and stale branches sit beside good ones.
- **Runs with approvals/sandbox bypassed** – fine for a trusted checkout, revisit for untrusted input.
- **An LLM verifier can miss errors or reject correct proofs** – why the loop leans on a strict JSON service + audit trail.

The natural-language verification boundary is shared by all four systems (closing slide) – not a Rethlas-specific gap.

# proofQED

[github.com/proofQED/QED](https://github.com/proofQED/QED) · [arXiv:2604.24021](https://arxiv.org/abs/2604.24021)

# proofQED: A Workflow Around Coding-Agent CLIs

A natural-language proof-search system built around general coding-agent CLIs (Claude, Codex, Gemini).

- the file system holds state;
- prompts define each role's contract;
- output is a natural-language `proof.md`.

The contribution is the **workflow**, not a new prover: it orchestrates general coding agents into a decompose → prove → verify → regulate loop.

Natural-language by design: `proof.md` is reviewed by the LLM verifier, with human experts checking some cases.

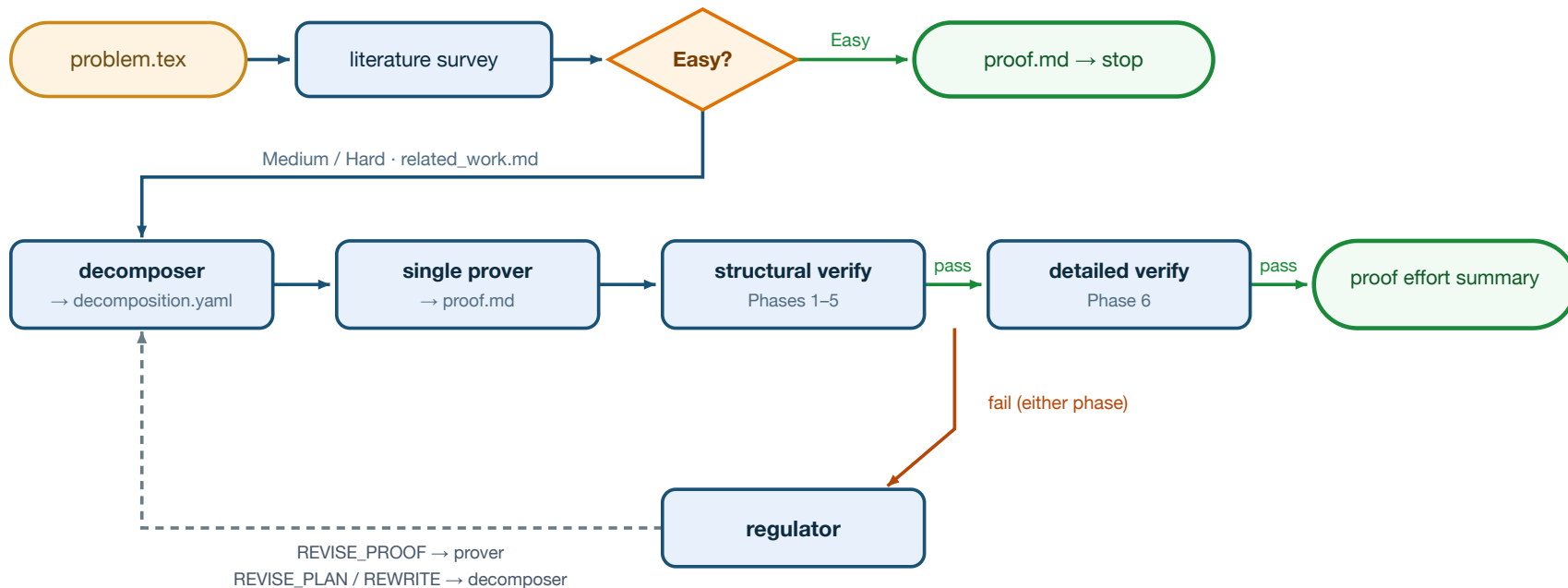
Local analysis: `casestudy/nlagents/QED/README.md`. Only mode: `prover.mode: "decomposition"` (the earlier flat "simple" mode was removed).

# The proofQED Pipeline

□ agent / stage

□ file

□ decision



Structural and detailed verification are split across separate slides; the pipeline view collapses them into one node.

# Difficulty Triage and the Easy Short-Circuit

Stage 0's survey agent does two jobs in one call: it classifies the problem, and **only if Easy** it also writes the final `proof.md` and stops – the Easy proof is a side effect of the survey call, not a prover stage.

```
difficulty_evaluation.md (written first):  
  ## Classification: Easy / Medium / Hard  
  Easy = "I can one-shot this, no literature."  
  prompt biases away from Easy:  
    any doubt → Medium/Hard (decomposer is safer).  
  
_parse_difficulty(): first line with "classification",  
  uppercase, test EASY → MEDIUM → HARD in order.  
short-circuit fires iff  
  difficulty = "easy" AND proof.md non-empty.
```

Medium vs Hard take the **same** path; the label only changes survey depth. At the control flow, triage is binary: Easy vs not-Easy.

On the Easy path the proof comes straight from the survey call and skips the later verifiers – triage doubles as cost control: spend the full decompose/verify/regulate loop only on the problems that need it.



# decomposition.yaml = the Proof Tree

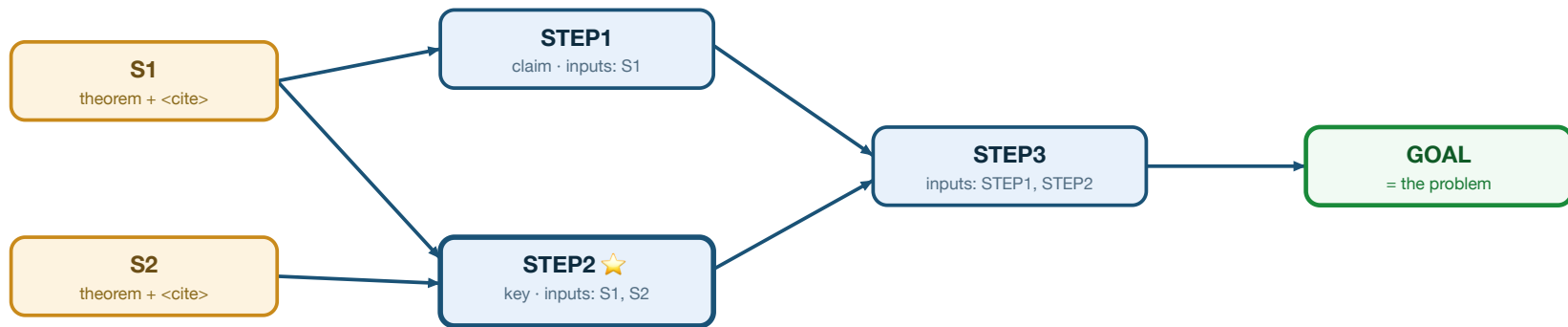
□ source (leaf)

□ step (claim)

□ key step

□ GOAL (root)

arrow = an *inputs* edge



**proof\_order** (topological): S1 · S2 → STEP1 → STEP2 → STEP3 → GOAL

One DAG, written once in `decomposition.yaml` (nodes + `inputs` edges + `proof_order`) and realized in `proof.md` node-for-node – a `### STEP_ID:` section per node with **Claim / Proof / Dependencies**.

Written by the decomposer LLM (agentic file-write); Python only `yaml.safe_loads` it – DAG legitimacy (topological order, `inputs`, `GOAL`) is checked by the structural verifier, not by code.

# Structural Verifier – Phases 1-5 (cheap, fatal)

|              |                                                                                                                                                                             |
|--------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| P1 integrity | proof's claim vs problem.tex, WORD-BY-WORD: a changed quantifier, weakened hypothesis, restricted domain, or special case → FAIL                                            |
| P2 complete  | every question / part answered + genuine proof work (not citation-listing); even an OPENLY ACKNOWLEDGED hole → FAIL                                                         |
| P3 citations | each <cite>: open the URL, locate via verifier_locator, match the statement WORD-BY-WORD, check usage; one FAIL cite → phase FAIL                                           |
| P4 adherence | coverage table per plan step (header? claim? proof? deps?); key steps tagged + rigorous; 4f: independently judge any step the prover declared false / impossible / circular |
| P5 rules     | check the proof against human-written global rules, if any                                                                                                                  |

These are **structural foundations only**: Phases 1-5 never check whether a step's math is correct (that is Phase 6). They run on the assembled `proof.md` as a standalone document, and a structural FAIL skips Phase 6 entirely.

# Detailed Verifier – Phase 6 (only if 1–5 passed)

Reads the structural report first; it does **not** re-check citations or adherence, and treats a cite that Phase 3 failed as poisoning every step that uses it.

|                 |                                                                                                                                                                                       |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 6a step-by-step | per step: claim · deps used correctly? · logical validity ·<br>math correctness · completeness; verify with SymPy / Z3 / NumPy;<br>verdict PASS / FAIL / UNCERTAIN (UNCERTAIN = fail) |
| 6b key steps    | each is_key_step <key-original-step>-tagged + rigorous; flag an<br>UNTAGGED nontrivial step – the prover may be hiding a weak argument                                                |
| 6c chain        | each step actually uses its declared deps; steps chain to GOAL                                                                                                                        |
| 6d coverage     | all cases + boundary / degenerate cases + every problem hypothesis                                                                                                                    |
| 6e assembly     | notation consistency · valid transitions · no dangling references                                                                                                                     |

Output is a per-step verdict table + an overall PASS/FAIL. "Almost right" is FAIL; a hard problem proved "easily" is treated as suspicious.

# How the Verifier Prompts Are Built

P1-P5 and 6a-6e are **not code** – they live entirely in `proof_verify_structural.md` / `proof_verify_detailed.md`. Python only templates paths + spawns the agent; the phases "exist" only because each prompt's fixed output template demands a PASS/FAIL for every one.

**Shared scaffolding:** a "very mean and critical", strict persona where **uncertain**  $\Rightarrow$  **fail**; agentic (read files, run tools, write the report via tool call); and a fixed output template that forces a verdict per phase / per step.

**Structural prompt** – scope-fenced: "ONLY the five phases", **do NOT check step math**; ordered cheapest  $\rightarrow$  fatal; each phase has an explicit FAIL trigger; 4f independently judges refuted plan steps.

**Detailed prompt** – runs only after structural PASS, reads the structural report (no re-checking cites); "Number 1 rule: every step justified"; SymPy/Z3 (contradiction  $\Rightarrow$  FAIL); "almost right" is still FAIL.

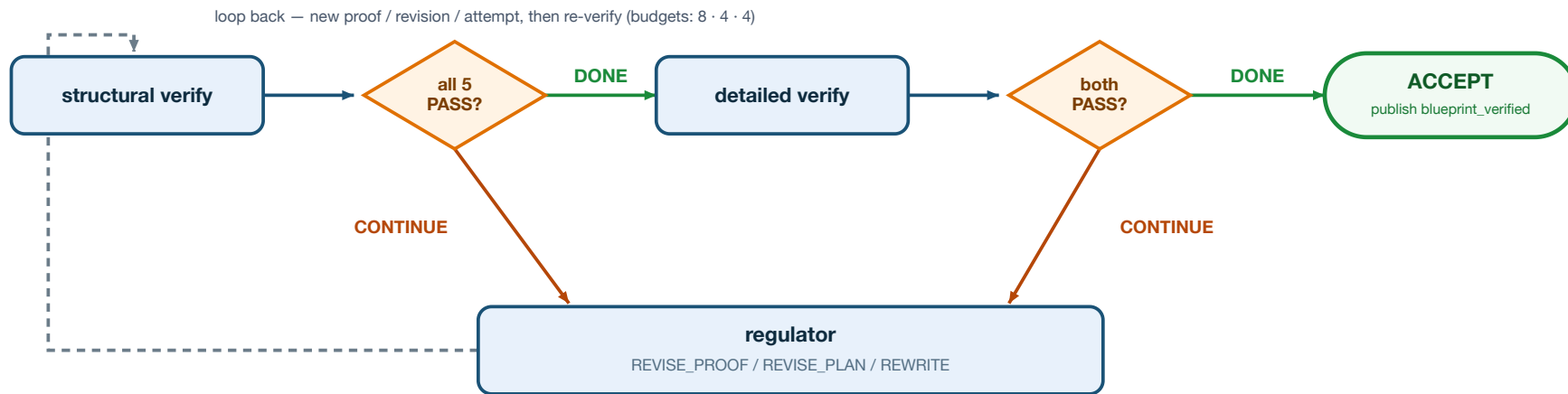
# How the Verifiers Run: an Agent, Not a Formal Checker

```
run_structural_verification() / run_detailed_verification():  
  load_prompt(proof_verify*.md + problem / proof / decomposition)  
  run_model_for_agent(role) → spawn the vendor CLI as a subprocess  
    (default codex exec --dangerously-bypass-approvals-and-sandbox)  
  agent reads files, may run SymPy / Z3, writes the report.md ITSELF  
  Python: no file written? dump stdout; then return the report PATH
```

The agent's job **is** to check the proof – but **informally**: it reads, reasons, and may run SymPy/Z3 spot-checks, then writes a critique. It is **not** a formal / symbolic / kernel checker, and not a dedicated program – the same generic agent call as the decomposer and prover, only the prompt differs.

The **detailed verifier** uses the same mechanism but runs **only if the structural verdict is DONE**, and is handed the structural report – so it does **not** re-check citations or adherence; a cite that Phase 3 failed poisons every step using it.

# Verdict Flow: DONE Advances, CONTINUE Loops



**DONE** = the verdict agent read the report and saw Overall Verdict **PASS** (structural: all 5 phases; final: both reports, no unresolved issues). **CONTINUE** = any FAIL / missing / "when in doubt" → the report + which-phase-failed go to the regulator, which picks the retry level; the loop repeats until DONE or a budget runs out.

# The Regulator: REVISE\_PROOF / REVISE\_PLAN / REWRITE

## REVISE\_PROOF

Plan is sound; rewrite the proof. `proof_N`

## REVISE\_PLAN

Plan has a structural problem; edit it. `revision_N`

## REWRITE

Strategy is wrong; new plan. `attempt_N`

```
budgets (config.yaml):  
  max_proof_attempts  8  
  max_revisions       4  
  max_decompositions  4
```

The regulator receives `verification_phase`: a structural failure points at the plan, a detailed failure at the execution. It is still an LLM and can misattribute.

# plan\_history: Cross-Attempt Failure Memory

```
on REVISE_PLAN / REWRITE the regulator  
appends to plan_history.md:  
  the failed strategy (one line)  
  the verbatim key step  
  why the verifier rejected it  
  routes not to retry  
  parts still reusable  
  advice for the next decomposer
```

**Read by the decomposer** – mandatory in REVISE / REWRITE: the next plan must avoid every route listed under "Do NOT try again". (The regulator also reads the current file before it appends.)

**Written by the regulator agent** (its own file tool), only on REVISE\_PLAN / REWRITE – a REVISE\_PROOF keeps the plan, so nothing is abandoned. Python just seeds the empty file. Same idea as Rethlas's `failed_paths`, at the plan/strategy level.



# CLI Orchestration and Per-Agent Models

one subprocess per agent call, scraping JSON:

```
claude -p --output-format json --model M <prompt>
codex --search -m M exec --json
      --dangerously-bypass-approvals-and-sandbox
gemini -m M --approval-mode yolo -o json -p <prompt>
```

prompt = one argv element; no model runtime of its own

Each CLI has its **own JSON parser**; on a parse failure all three fall back to **raw stdout** as the answer – a vendor format change breaks it silently.

reliability is per-provider, not per-role:

|               |                 |                           |
|---------------|-----------------|---------------------------|
| retries       | Claude          | 3x [30/60/120s]           |
|               | Codex / Gemini: | none                      |
| exit $\neq$ 0 | Claude          | retry $\rightarrow$ fatal |
|               | Codex           | warning if parsed         |
|               | Gemini          | always fatal              |

8 roles can each pick a provider+model – but the shipped `config.yaml` sets **every role to codex**, so solver and verifiers are the same family by default.

---

All three bypass approvals/sandbox (full host access) – fine for a trusted checkout; the Claude wrapper also strips provider env vars to stop cross-contamination.

# Structured Tags: Who Writes, Who Reads

| marker                                 | what it marks                            | written by                                                       | read / checked by                                                                    |
|----------------------------------------|------------------------------------------|------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| <code>### STEPn + Dependencies</code>  | the tree: one section per plan node      | <b>prover</b>                                                    | structural <b>Phase 4</b> (coverage table); detailed <b>Phase 6</b> finds each step  |
| <code>&lt;cite&gt;</code>              | an external result + its exact statement | <b>decomposer</b> (in the plan) and <b>prover</b> (in the proof) | structural <b>Phase 3</b> re-verifies it: real? faithful?                            |
| <code>&lt;key-original-step&gt;</code> | the hard core of a key step              | <b>prover</b>                                                    | structural <b>4c</b> (is it tagged?) + detailed <b>Phase 6</b> (rigor / hand-waving) |
| <code>&lt;heuristics&gt;</code>        | why that key step should hold            | <b>prover</b>                                                    | structural <b>4c</b> (is it present?)                                                |

All four are **prompt-level conventions** – no Python parses them; the LLM verifiers read them. They make the proof's hidden structure explicit so verification has a checklist: *how steps depend on each other* (Dependencies), *what comes from the literature* (`<cite>`), and *which step is the hard one* (`<key-original-step>`). The `<subgoal>` tags are archived, not live.

# proofQED Limitations

- **Easy triage** rests on the model's own confidence call – a self-declared "I can one-shot this."
- **High token cost:** many CLI calls across decomposer, prover, two verifiers, regulator.
- **Exact-match citation rule** cuts hallucination but is unlike natural math citing.
- These are **workflow trade-offs**, not capability ceilings.

**Length is not a barrier.** QED has produced a **63-page** proof generated entirely by the system, no expert human intervention – the advection-diffusion lower bound in An & Xu, arXiv:2605.20623.

proofQED is a research-proof candidate generator with structured self-review – its job is to produce a strong, auditable natural-language proof and the evidence behind it.

# Moonshine

[github.com/DeepMathLLM/Moonshine](https://github.com/DeepMathLLM/Moonshine)

# The Moonshine System

Broader than one proof attempt: Moonshine keeps the generation/verification loop running across **many turns** and stores the whole research process on disk – a local math-research **workbench**.

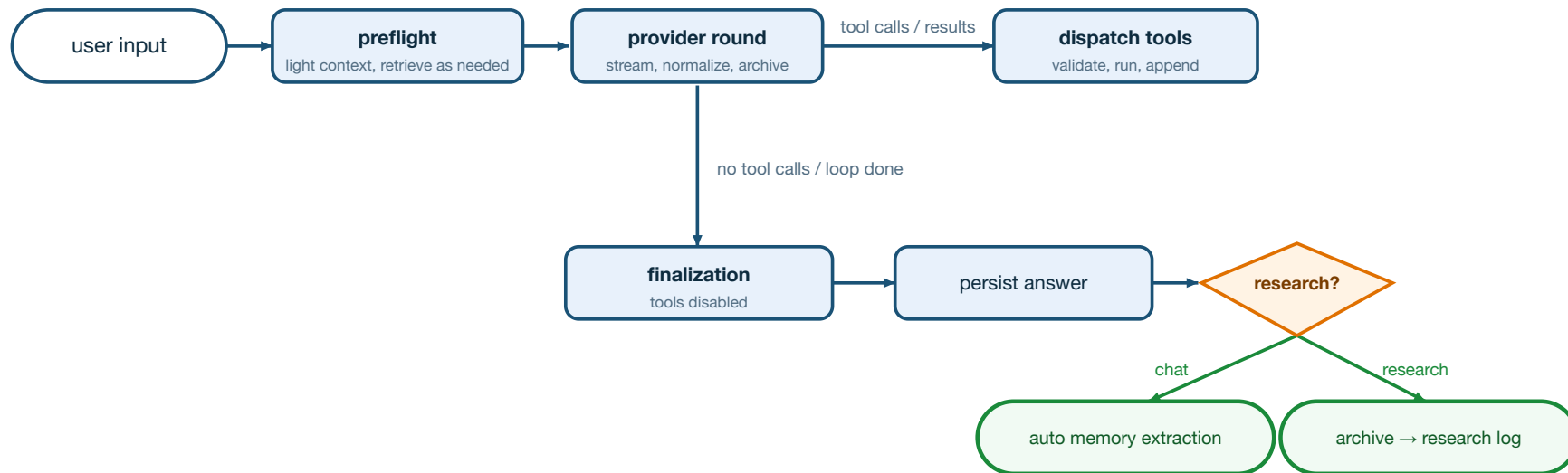
A local-first **CLI agent harness**: an LLM tool-calling loop wrapped in persistent project memory, a Markdown skill system, a research log, and LLM-review verification (state in `~/.moonshine`).

```
projects/<slug>/
  AGENTS.md  rules.md
  workspace/ problem.md
              blueprint(_verified).md
  references/ notes papers surveys
  memory/    research_log.jsonl + .md
              + sqlite index    by_type/
```

It is the **harness**, not the model: a tool-calling loop plus persistence around whichever LLM you plug in, not new model weights or a one-shot chat CLI.

Two `AGENT.md` profiles: `moonshine-core` (chat) and `research-control-loop` (the autonomous researcher that drives the autopilot).

# The Moonshine Turn



Research mode wraps this turn in an autopilot that repeats until budget reached, provider offline, two identical turns, or `verify_overall(final)` passes.

# Skills as Markdown Documents

Skills are `SKILL.md` files (frontmatter + sections).  
Only a compact **skill index** is in the system  
prompt; `load_skill_definition(slug)` pulls the  
full body (audited as `skill_load`).

`manage_skill` can create / edit / patch / delete  
installed skills (validated, logged). Loading is  
prompt-level – not forced.

|                   |                                                          |
|-------------------|----------------------------------------------------------|
| research design   | problem-generator / -refiner                             |
| proof exploration | problem-solver, lemma-prover,<br>proof-constructor       |
| examples          | toy / counterexamples                                    |
| failure analysis  | identify-key-failures,<br>proof-corrector, strengthening |
| memory/retrieval  | query-memory, consolidation                              |
| verification      | verify-overall + 3 reviewers                             |

# Tools, Modes, and MCP

Native tools declared in Markdown (JSON metadata) → `ToolRegistry` → Python handlers:  
`query_memory`, `search_knowledge`,  
`verify_overall`, `load_skill_definition`, ...

`MODE_HIDDEN_TOOLS` per mode: research hides `manage_skill` + the manual write tools (`store_conclusion`, `add_knowledge`, `record_research_artifact`, ...) – so the autopilot owns the research log and skills are frozen mid-run.

MCP bridge: stdio JSON-RPC, env allowlist, credential redaction, FS root scoping – a local trusted runtime.



# Memory and the Research Log

three layers:

|                  |                                             |
|------------------|---------------------------------------------|
| session trace    | conversation + events                       |
| project memory   | research_log.jsonl<br>(+ md / sqlite views) |
| global knowledge | conclusions + index                         |

Append-only log; each record: `id`, `created_at`, `project`, `session`, `round`, `type`, `title`, `content`, `source_refs`. Failed paths + counterexamples are first-class. A `verified_conclusion` mirrors to global knowledge.

Six stores (SQLite + JSONL + Markdown + vector); the vector index is **local-first** – pure-Python cosine, `sha256` embedding fallback, runs offline.

Canonical source is the JSONL log; the Markdown view and SQLite/FTS index are derived.

# Multi-Dimension LLM Verification

`verify_overall(scope)` – **two nested levels** of pessimistic "any-failure" voting:

```
3 dimensions:
  assumption  assumptions used right?
  computation  calculations correct?
  logic        chain complete?
each dimension: N independent reviewers
  any reviewer  $\neq$  correct  $\rightarrow$  dimension fails
any dimension fails  $\rightarrow$  overall fails
```

The code calls it **pessimistic**: a claim passes only if **every** reviewer of **every** dimension returns `correct`. Failures carry de-duplicated `critical_errors` / `gaps` / `repair_hints`.

`verified_conclusion` = passed this review (verifier + solver both LLMs  $\rightarrow$  correlated errors); `scope="final"` is the autopilot's gate; an offline provider skips it.

# Moonshine Limitations

- **Many persistence layers** (SQLite, JSONL, Markdown, archive) → consistency burden.
- **Skill loading + research log** depend on the model obeying the prompt.
- **manage\_skill** lets the model alter installed skills – worth review.
- **Hand-written provider adapters**; non-standard package layout (`config_legacy.py` breaks `py_compile`).
- **Cross-turn state on disk** is powerful but makes exact reruns harder.

Moonshine's distinctive bet is breadth: a persistent multi-turn research workbench rather than a single proof attempt.

# Each System's Additions

## IMO25 – the loop

roles by prompt, separable artifact, actionable verdict, repair, accept on repeated agreement; bounded / parallel / resumable.

## Rethlas – the environment

workspaces, explicit contracts, code guardrails, searchable memory (failed\_paths), named skills, /verify service, audit trail.

## proofQED – the workflow

difficulty triage + Easy short-circuit, explicit plan, two-level verification, regulator attribution, plan\_history.

## Moonshine – the workbench

the loop across many turns: persistent project memory + research log, Markdown skills, multi-dimension LLM review.

Each system reuses the loop and adds a layer: the loop itself, an environment around each role, an organized multi-stage workflow, then a persistent research workbench across turns.

# One Loop, Realized Differently

|                    | IMO25                 | Rethlas                           | proofQED                            | Moonshine                                |
|--------------------|-----------------------|-----------------------------------|-------------------------------------|------------------------------------------|
| <b>structure</b>   | whole solution        | subgoal plans + branches          | <code>decomposition.yaml</code> DAG | skills, no fixed plan                    |
| <b>parallelism</b> | best-of-n attempts    | sub-agent per plan                | sequential revise / rewrite         | sequential autopilot                     |
| <b>verify</b>      | 5× yes/no, same model | JSON <code>/verify</code> service | structural → detailed               | 3-dim pessimistic                        |
| <b>memory</b>      | none (in-loop)        | <code>failed_paths</code> channel | <code>plan_history</code>           | research log + <code>~/.moonshine</code> |
| <b>guardrails</b>  | all prompt            | some code, mostly prompt          | mostly prompt                       | hidden-tools + schema in code            |

loop → environment → workflow → workbench: each adds machinery the last lacked – and none crosses the natural-language verification boundary.

# The Natural-Language Boundary

all four are natural-language systems:

"accepted" = an AI verifier reviewed the proof and approved it  
that is a careful review, not a mechanical proof-kernel check

This is the design space, not a defect: these systems target informal, research-level math an autoformalizer can't yet reach. The open question is how to make AI acceptance trustworthy.

Each answers it differently: IMO25 with repetition (5 passes); Rethlas with a strict JSON service + audit trail; proofQED with a two-level gate + failure memory; Moonshine with multi-dimension review.

A verifier is itself a stochastic model that can be wrong both ways – so the mitigation, not the kernel, is each system's real contribution.